14. Bash Scripting Fundamentals

14. Bash Scripting Fundamentals

Phase: Phase 5 — Shell scripting
Estimated time: 2 days
Slug: `14-bash-scripting-intro`

Learning objectives

- Write scripts with shebang and set -euo pipefail
- Use variables, quotes, and arrays basics
- Write if/for/while
- Make scripts executable and reusable

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Every sysadmin task you type twice deserves to become a script. A script is just a text file of shell commands, but it adds three things the interactive prompt lacks: repeatability, reviewability, and the ability to run unattended (cron, systemd, CI). Bash is on nearly every Linux host, so a small bash script is the cheapest automation you can ship.

The danger is that bash is forgiving in the wrong ways. An unset variable silently expands to nothing, a failed command does not stop the script, and an unquoted value splits into several words. This lesson builds the habits that keep tiny scripts predictable: a shebang, quoting, positional parameters, and default values such as `${1:-world}`.

Core concepts

Concept	Syntax	Meaning
Shebang	`#!/usr/bin/env bash`	Tells the kernel which interpreter runs the file
Executable bit	`chmod +x hello.sh`	Lets you run `./hello.sh` directly
Variable	`name=Ada` then `"$name"`	No spaces around `=`; quote on use
Positional args	`$1 $2 ... $#`	Arguments passed to the script
All args	`"$@"`	Each argument kept as a separate word
Default value	`${1:-world}`	Use `$1`, or `world` if unset or empty
Command substitution	`$(date +%F)`	Replace with the command output
Exit status	`$?`	0 means success, non-zero means failure

How a script runs:

```
you type: ./hello.sh Ada
      |
      v
kernel reads first line  (#!/usr/bin/env bash)
      |
      v
bash starts, receives file + args
      |        $0=./hello.sh  $1=Ada  $#=1
      v
commands run top to bottom
      |
      v
last command's status becomes the script's exit code
```

Quoting rules in one line: single quotes keep text literal, double quotes allow `$var` and `$(cmd)` to expand but prevent word splitting, and no quotes means the shell splits on whitespace and expands globs.

Worked examples

Create a script that greets a name, defaulting to `world`:

```
cat > hello.sh <<'EOF'
#!/usr/bin/env bash
name=${1:-world}
echo "Hello, $name"
EOF
chmod +x hello.sh
./hello.sh
./hello.sh Ada
./hello.sh "Ada Lovelace"
```

Sample output:

```
Hello, world
Hello, Ada
Hello, Ada Lovelace
```

The same idea inside a function, which is what the exercise uses:

```
hello() {
  local name=${1:-world}
  echo "Hello, $name"
}
hello
hello Linus
```

Sample output:

```
Hello, world
Hello, Linus
```

Difference between `:-` and `-`:

```
unset x; echo "[${x:-fallback}]"
x=""; echo "[${x:-fallback}]"
x=""; echo "[${x-fallback}]"
```

Sample output:

```
[fallback]
[fallback]
[]
```

Checking the exit status and counting arguments:

```
./hello.sh a b c; echo "status=$?"
set -- one two three; echo "count=$#"
```

Sample output:

```
Hello, a
status=0
count=3
```

Mini-lab

1. Create a directory `~/lab14` and `cd` into it.
2. Write `hello.sh` with the shebang, `name=${1:-world}`, and an `echo`.
3. Run it with `bash hello.sh` first, then `chmod +x` and run `./hello.sh`.
4. Run it with no argument, one argument, and a quoted two-word argument.
5. Remove the quotes from `"$name"` in the echo and rerun with `"Ada Lovelace"`; note how the spaces collapse. Restore the quotes.
6. Add `echo "args: $#"` and confirm the count for each run.
7. Run `bash -n hello.sh` (syntax check) and `bash -x ./hello.sh Ada` (trace) and read the trace output line by line.
8. Move the greeting into a `hello()` function and call it from the bottom.

Decision guide / failure modes

Symptom	Likely cause	Fix
`Permission denied` running `./hello.sh`	No executable bit	`chmod +x hello.sh`
`bad interpreter: No such file`	Wrong shebang or CRLF line endings	Fix path; convert with `dos2unix`
`name: command not found`	Space around `=` (`name = Ada`)	Write `name=Ada`
Output loses spaces	Unquoted variable	Use `"$name"`
Greeting is `Hello,`	Variable empty, no default	Use `${1:-world}`
Works in terminal, fails in cron	Different PATH and cwd	Use absolute paths

When to use bash: glue between commands, file and process orchestration, small admin tasks. When to switch language: heavy parsing, JSON, data structures, or anything over a couple of hundred lines; Python or Go will be safer there.

Rules of thumb: always start with a shebang, quote every expansion, prefer `${var:-default}` over testing in an `if`, and run `bash -n` and `bash -x` before trusting a script.

Common mistakes

- Unquoted variables causing word-splitting bugs
- Omitting set -e and missing failures
- Parsing ls in scripts

Exercises

Work in `exercises/14-bash-scripting-intro/`. Read that folder's `README.md` fully before coding.

```
cd exercises/14-bash-scripting-intro
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open exercise.sh and finish the hello function. It must print exactly "Hello, " on one line, using the first argument as the name. When no argument is given (or it is empty), the name must be "world". Keep the local name variable that is already there and use the ${1:-world} default style from the lesson. Quote the variable when you print it, so a name with spaces such as "Ada Lovelace" is kept intact. Do not print anything else: no extra spaces, no trailing text. Run ./test.sh until every check passes, then run the function by hand with no argument, one argument, and a two-word quoted argument.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/14-bash-scripting-intro/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does set -euo pipefail do?
2. Why quote "$var"?
3. Shebang purpose?

Further reading

- https://mywiki.wooledge.org/BashGuide